

**LAPORAN TEKNIS AKHIR PROYEK BERBASIS MASALAH
MATA KULIAH KEAMANAN INFORMASI (KOM1315)**

**IMPLEMENTASI KEAMANAN INFORMASI BERBASIS
PROTOKOL AAA,
ENKRIPSI DATA, DAN TANDA TANGAN DIGITAL
PADA APLIKASI TUMBUH**

Kelompok 9



Rafif Muhammad Farras (NIM: _____)

Mochamad Chairulridjal (NIM: _____)

[Nama Anggota 3] (NIM: _____)

[Nama Anggota 4] (NIM: _____)

**PROGRAM STUDI ILMU KOMPUTER
SEKOLAH SAINS DATA, MATEMATIKA, DAN INFORMATIKA
INSTITUT PERTANIAN BOGOR
BOGOR
2026**

RINGKASAN

KELOMPOK 9. Implementasi Keamanan Informasi Berbasis Protokol AAA, Enkripsi Data, dan Tanda Tangan Digital pada Aplikasi TUMBUH.

TUMBUH adalah aplikasi web *career and internship tracker* bagi mahasiswa IPB, HR perusahaan, dan admin platform. Karena sistem ini mengelola data pribadi mahasiswa, dokumen CV, surat lamaran, serta status seleksi, diperlukan kontrol keamanan yang menyeluruh untuk menjaga kerahasiaan, integritas, dan akuntabilitas data. Laporan ini menyajikan rancangan dan implementasi kontrol keamanan informasi pada TUMBUH dengan menerapkan kerangka AAA (*Authentication, Authorization, Accounting*) yang dilengkapi kriptografi simetris, *password hashing*, dan tanda tangan digital.

Implementasi mencakup autentikasi berbasis kata sandi (bcrypt dengan *salt* otomatis), JWT (HS256), dan Google OAuth; otorisasi berbasis peran (*role-based access control*) beserta pemeriksaan kepemilikan data dan kontrol akses CV; serta akuntansi melalui layanan *audit logging* mikroservis yang mengamankan integritas catatan menggunakan rantai *hash* SHA-256. Kerahasiaan data sensitif dijaga dengan enkripsi tingkat kolom (Fernet, AES-128-CBC + HMAC-SHA256), sedangkan *non-repudiation* pada peristiwa lamaran dijamin dengan tanda tangan digital Ed25519. Seluruh modul kriptografi terintegrasi langsung pada alur utama aplikasi, bukan skrip terpisah, dan seluruh kunci dikelola melalui variabel lingkungan tanpa *hardcoded keys*.

Pengujian unit terhadap fungsi keamanan menunjukkan seluruh kasus uji berhasil (9 dari 9), meliputi verifikasi *roundtrip* enkripsi-dekripsi, deteksi pemalsuan tanda tangan, penolakan akses oleh pengguna tanpa hak (HTTP 403), serta penolakan dekripsi terhadap *ciphertext* yang dirusak. Hasil ini menunjukkan bahwa kontrol keamanan TUMBUH telah memenuhi sasaran kerahasiaan, integritas, dan akuntabilitas.

Kata kunci: AAA, enkripsi, keamanan informasi, tanda tangan digital, TUMBUH

DAFTAR ISI

DAFTAR TABEL	v
DAFTAR GAMBAR	v
DAFTAR LAMPIRAN	v
I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan	2
1.4 Manfaat	2
1.5 Ruang Lingkup	2
II TINJAUAN PUSTAKA	3
2.1 Keamanan Informasi dan Segitiga CIA	3
2.2 Protokol AAA	3
2.3 Fungsi Hash dan Password Hashing	3
2.4 Kriptografi Simetris dan Fernet	4
2.5 Kriptografi Kunci Publik dan Tanda Tangan Digital	4
2.6 JSON Web Token dan OAuth 2.0	4
2.7 Aplikasi TUMBUH	5
III METODE	6
3.1 Tahapan Pelaksanaan	6
3.2 Alat dan Bahan	6
3.3 Arsitektur Keamanan Sistem	6
3.4 Pemodelan Ancaman dan Pemetaan Kontrol	7
3.5 Manajemen Kunci	7
3.6 Prosedur Pengujian	8
IV HASIL DAN PEMBAHASAN	9
4.1 Authentication	9
4.2 Authorization	9
4.3 Accounting (Audit Logging)	10

4.4	Password Hashing dan Salting	10
4.5	Enkripsi dan Dekripsi Data	11
4.6	Tanda Tangan Digital dan Non-Repudiation	12
4.7	Kontrol Keamanan Pendukung	12
4.8	Integrasi pada Alur Utama Aplikasi	13
4.9	Hasil Pengujian	13
V	SIMPULAN DAN SARAN	15
5.1	Simpulan	15
5.2	Saran	15
	DAFTAR PUSTAKA	16
	LAMPIRAN	17

DAFTAR TABEL

1	Perangkat lunak dan pustaka utama yang digunakan	6
2	Pemetaan ancaman terhadap kontrol keamanan pada TUMBUH	7
3	Ringkasan fitur keamanan TUMBUH dan teknologi penyusunnya	13
4	Ringkasan hasil pengujian fungsi keamanan	14

DAFTAR GAMBAR

1	Arsitektur keamanan aplikasi TUMBUH yang mencakup data <i>at rest</i> , <i>in transit</i> , dan manajemen kunci	7
---	---	---

DAFTAR LAMPIRAN

1	Lampiran 1 Keluaran pelaksanaan pengujian unit fungsi keamanan	18
2	Lampiran 2 Struktur folder repositori proyek	19

I PENDAHULUAN

1.1 Latar Belakang

Transformasi digital pada layanan karier dan magang mendorong munculnya platform yang menghubungkan mahasiswa, perusahaan, dan pengelola institusi dalam satu kanal daring. TUMBUH merupakan aplikasi web *career and internship tracker* yang dikembangkan untuk membantu mahasiswa IPB menemukan lowongan kerja dan magang, menyimpan lowongan, serta mengirim lamaran. Pada sisi lain, HR perusahaan dapat mengelola profil perusahaan, membuat lowongan, meninjau pelamar, dan memperbarui status seleksi, sedangkan admin platform mengawasi data pengguna dan aktivitas sistem.

Sistem dengan karakteristik tersebut menyimpan dan memproses data bernilai sensitif, seperti identitas dan profil akademik mahasiswa, nomor induk mahasiswa, dokumen CV, surat lamaran, serta status seleksi pelamar. Pengelolaan data sensitif oleh banyak peran pengguna menimbulkan risiko keamanan yang nyata, antara lain pencurian akun, akses tidak sah terhadap data milik orang lain, manipulasi status lamaran, kebocoran data pribadi, dan penyangkalan aktivitas (*repudiation*) ketika terjadi insiden. Tanpa kontrol keamanan yang dirancang dengan baik, sistem menjadi rentan terhadap penyalahgunaan dan kehilangan kepercayaan pengguna.

Keamanan informasi umumnya diarahkan untuk memenuhi tiga sasaran utama, yaitu kerahasiaan (*confidentiality*), integritas (*integrity*), dan ketersediaan (*availability*) yang dikenal sebagai segitiga CIA (Stallings dan Brown 2017). Untuk mencapai sasaran tersebut pada sistem multi-peran, kerangka AAA (*Authentication, Authorization, Accounting*) menjadi pendekatan yang lazim digunakan: autentikasi memverifikasi identitas, otorisasi membatasi akses sesuai hak, dan akuntansi mencatat aktivitas sebagai jejak audit (Vollbrecht et al. 2000). Kerangka ini perlu diperkuat dengan mekanisme kriptografi seperti *password hashing*, enkripsi data, dan tanda tangan digital agar data tetap terlindungi baik saat disimpan maupun saat dipertukarkan.

Laporan teknis ini disusun sebagai luaran akhir Proyek Berbasis Masalah (*Problem-Based Learning*) mata kuliah Keamanan Informasi (KOM1315). Laporan mendokumentasikan rancangan, implementasi, dan pengujian kontrol keamanan pada aplikasi TUMBUH, dengan penekanan pada integrasi protokol AAA, enkripsi data sensitif, tanda tangan digital, serta pengelolaan kunci yang aman.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, permasalahan yang diangkat dirumuskan sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan kerangka AAA (*Authentica-*

tion, Authorization, Accounting) pada aplikasi TUMBUH?

2. Bagaimana menjamin kerahasiaan data sensitif pengguna melalui *password hashing* dan enkripsi data?
3. Bagaimana menjamin integritas dan *non-repudiation* terhadap peristiwa penting pada sistem melalui tanda tangan digital dan jejak audit?
4. Bagaimana memverifikasi bahwa kontrol keamanan yang dibangun bekerja sesuai sasaran melalui pengujian?

1.3 Tujuan

Tujuan penyusunan laporan ini adalah:

1. menerapkan kontrol *Authentication, Authorization, dan Accounting* yang terintegrasi pada alur utama aplikasi TUMBUH;
2. menerapkan *password hashing* berbasis bcrypt dan enkripsi tingkat kolom berbasis Fernet untuk melindungi data sensitif;
3. menerapkan tanda tangan digital Ed25519 dan rantai *hash* SHA-256 guna menjamin integritas data dan *non-repudiation*;
4. menguji fungsi-fungsi keamanan tersebut serta mendokumentasikan hasilnya sebagai bukti pengujian.

1.4 Manfaat

Manfaat yang diharapkan adalah tersedianya rancangan dan implementasi kontrol keamanan informasi yang dapat dijadikan acuan penerapan keamanan pada aplikasi web multi-peran sejenis. Secara praktis, hasil ini meningkatkan perlindungan data pengguna TUMBUH dan memberikan jejak audit yang dapat dipertanggungjawabkan. Secara akademik, kegiatan ini memperkuat pemahaman penerapan konsep AAA, kriptografi simetris, dan kriptografi kunci publik dalam konteks pengembangan perangkat lunak nyata.

1.5 Ruang Lingkup

Ruang lingkup laporan dibatasi pada aspek keamanan informasi aplikasi TUMBUH dan tidak membahas keseluruhan fungsionalitas bisnis aplikasi secara rinci. Kontrol keamanan yang dibahas mencakup autentikasi, otorisasi, akuntansi/audit, *password hashing*, enkripsi tingkat kolom, tanda tangan digital, *security headers*, *rate limiting*, CORS, keamanan unggahan berkas, verifikasi surel, serta manajemen rahasia (*secret management*). Implementasi dilakukan pada arsitektur tiga komponen, yaitu *frontend* berbasis React, *backend* berbasis FastAPI, dan layanan audit berbasis Express, dengan basis data PostgreSQL.

II TINJAUAN PUSTAKA

2.1 Keamanan Informasi dan Segitiga CIA

Keamanan informasi adalah upaya melindungi informasi dan sistem informasi dari akses, penggunaan, pengungkapan, gangguan, modifikasi, atau kerusakan yang tidak sah. Sasaran keamanan informasi secara klasik dirangkum dalam segitiga CIA, yaitu kerahasiaan (*confidentiality*), integritas (*integrity*), dan ketersediaan (*availability*) (Stallings dan Brown 2017). Kerahasiaan menjamin informasi hanya dapat diakses oleh pihak yang berwenang; integritas menjamin informasi tidak diubah secara tidak sah; dan ketersediaan menjamin informasi dapat diakses ketika dibutuhkan. Pada sistem multi-peran, sasaran ini sering diperluas dengan akuntabilitas (*accountability*) dan *non-repudiation* agar setiap aktivitas dapat dilacak dan tidak dapat disangkal.

2.2 Protokol AAA

AAA merupakan kerangka kerja keamanan yang terdiri atas tiga fungsi (Vollbrecht et al. 2000). *Authentication* adalah proses verifikasi identitas pengguna, misalnya melalui kombinasi nama pengguna dan kata sandi atau melalui penyedia identitas pihak ketiga. *Authorization* adalah proses penentuan hak akses pengguna terhadap sumber daya tertentu setelah identitasnya terverifikasi. *Accounting* adalah proses pencatatan aktivitas pengguna, yang menjadi dasar audit, deteksi anomali, dan pelacakan insiden. Penerapan AAA secara konsisten menjadikan kontrol akses lebih terstruktur dan mendukung kepatuhan terhadap kebijakan keamanan.

2.2.1 Role-Based Access Control

Role-based access control (RBAC) adalah model otorisasi yang memberikan hak akses berdasarkan peran yang melekat pada pengguna, bukan pada identitas individu secara langsung (Sandhu et al. 1996). Dengan RBAC, izin dikelompokkan ke dalam peran (misalnya mahasiswa, HR, dan admin), sehingga administrasi hak akses menjadi lebih sederhana dan konsisten. RBAC sering dilengkapi pemeriksaan kepemilikan (*ownership check*) untuk memastikan pengguna hanya dapat mengakses sumber daya miliknya sendiri.

2.3 Fungsi Hash dan Password Hashing

Fungsi *hash* kriptografis memetakan masukan berukuran sembarang menjadi keluaran berukuran tetap dengan sifat satu arah dan tahan tabrakan (Stallings dan Brown 2017). SHA-256 merupakan fungsi *hash* yang banyak digunakan untuk memeriksa integritas data. Untuk penyimpanan kata sandi, fungsi *hash* biasa tidak memadai karena rentan terhadap serangan kamus dan tabel pelangi. Oleh karena itu digunakan fungsi *password hashing* khusus seperti bcrypt yang bersifat *adaptive* dan menambahkan *salt* acak pada

tiap kata sandi (Provos dan Mazieres 1999). *Salt* memastikan dua kata sandi yang sama menghasilkan *hash* berbeda, sedangkan faktor biaya (*cost factor*) memperlambat proses sehingga serangan tebak paksa (*brute force*) menjadi mahal secara komputasi.

2.4 Kriptografi Simetris dan Fernet

Kriptografi simetris menggunakan satu kunci rahasia yang sama untuk enkripsi dan dekripsi. *Advanced Encryption Standard* (AES) adalah algoritma enkripsi blok simetris standar yang banyak digunakan (National Institute of Standards and Technology 2001). Untuk menjaga kerahasiaan sekaligus integritas data, enkripsi sering dipadukan dengan kode autentikasi pesan (*message authentication code*). Fernet adalah spesifikasi enkripsi simetris yang mengombinasikan AES-128 mode CBC untuk kerahasiaan dan HMAC-SHA256 untuk autentikasi, serta menyertakan *timestamp* dan *initialization vector* acak pada tiap token (Python Cryptographic Authority 2024). Dengan demikian, Fernet menyediakan *authenticated encryption* sehingga perubahan sekecil apa pun pada *ciphertext* akan terdeteksi saat dekripsi.

2.5 Kriptografi Kunci Publik dan Tanda Tangan Digital

Kriptografi kunci publik menggunakan sepasang kunci, yaitu kunci privat yang dirahasiakan dan kunci publik yang dapat disebar. Tanda tangan digital memanfaatkan pasangan kunci ini untuk menjamin autentisitas, integritas, dan *non-repudiation*: penanda tangan membubuhkan tanda tangan dengan kunci privat, dan siapa pun dapat memverifikasinya dengan kunci publik (Stallings dan Brown 2017). Ed25519 adalah skema tanda tangan digital berbasis kurva eliptik (Curve25519) yang menawarkan keamanan tinggi, ukuran tanda tangan ringkas (64 byte), kecepatan tinggi, sifat deterministik, serta ketahanan terhadap serangan *timing* (Bernstein et al. 2012). Sifat *non-repudiation* menjamin pihak yang menandatangani suatu peristiwa tidak dapat menyangkal keterlibatannya di kemudian hari.

2.6 JSON Web Token dan OAuth 2.0

JSON Web Token (JWT) adalah format ringkas untuk merepresentasikan klaim yang dipertukarkan antara dua pihak secara aman (Jones et al. 2015). JWT yang ditandatangani dengan algoritma HMAC-SHA256 (HS256) memungkinkan server memverifikasi keaslian token tanpa menyimpan status sesi di sisi server. OAuth 2.0 adalah kerangka otorisasi yang memungkinkan aplikasi memperoleh akses terbatas atas nama pengguna melalui penyedia identitas tepercaya (Hardt 2012). Pada autentikasi berbasis Google, token identitas yang diterbitkan penyedia diverifikasi tanda tangannya, audiensnya, dan masa berlakunya sebelum dipercaya oleh aplikasi.

2.7 Aplikasi TUMBUH

TUMBUH dibangun dengan arsitektur tiga komponen. *Frontend* menggunakan React dan Vite, *backend* menggunakan kerangka FastAPI berbasis Python dengan ORM SQLAlchemy, dan layanan audit menggunakan Express berbasis Node.js. Basis data yang digunakan adalah PostgreSQL. Pemilihan FastAPI memberikan keuntungan validasi masukan otomatis melalui Pydantic dan mekanisme *dependency injection* yang memudahkan penerapan kontrol otorisasi secara konsisten pada setiap *endpoint*.

III METODE

3.1 Tahapan Pelaksanaan

Pelaksanaan proyek mengikuti tahapan yang selaras dengan kontrak mata kuliah, mulai dari analisis sistem dan pemodelan ancaman, perancangan kontrol keamanan, implementasi, integrasi ke alur utama aplikasi, hingga pengujian. Tahap analisis mengidentifikasi aset bernilai pada TUMBUH dan ancaman yang relevan. Tahap perancangan menentukan kontrol keamanan untuk setiap aset dan ancaman. Tahap implementasi menuangkan rancangan menjadi kode pada *backend*, layanan audit, dan *frontend*. Tahap integrasi memastikan seluruh modul kriptografi menyatu pada alur utama aplikasi, bukan sebagai skrip terpisah. Tahap pengujian memverifikasi bahwa kontrol bekerja sesuai sasaran.

3.2 Alat dan Bahan

Implementasi menggunakan perangkat lunak dan pustaka yang diringkas pada Tabel 1. Pemilihan pustaka mempertimbangkan kematangan, dukungan komunitas, dan ketersediaan primitif kriptografi yang teruji.

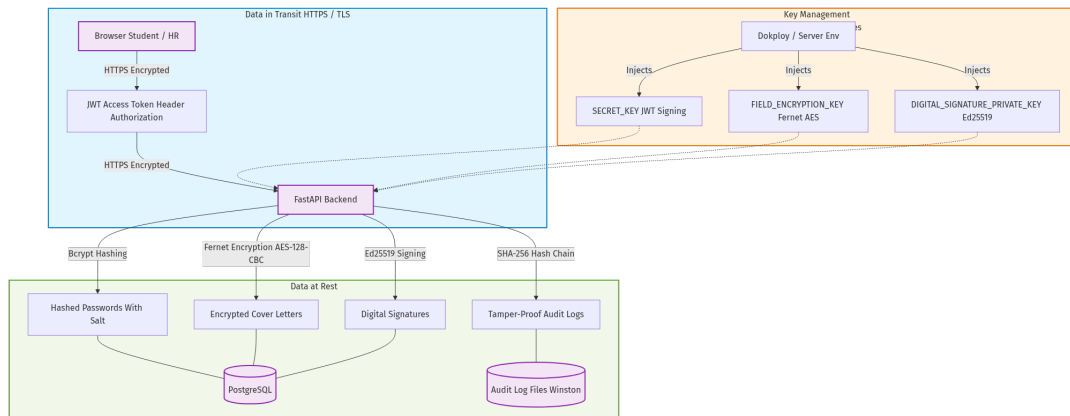
Tabel 1 Perangkat lunak dan pustaka utama yang digunakan

Komponen	Teknologi	Fungsi keamanan terkait
Frontend	React + Vite	Penyimpanan token, permintaan terautentikasi
Backend	FastAPI + SQLAlchemy	Logika AAA, validasi masukan, kriptografi
Audit	Express + Winston	Pencatatan audit dan rantai <i>hash</i>
Basis data	PostgreSQL	Penyimpanan data, <i>parameterized query</i>
Hashing	bcrypt 4.2.0	<i>Password hashing</i> dengan <i>salt</i>
Token	python-jose 3.3.0	Penerbitan dan verifikasi JWT (HS256)
Kriptografi	cryptography	Enkripsi kolom (Fernet) dan tanda tangan (Ed25519)
OAuth	google-auth 2.45.0	Verifikasi token identitas Google
Rate limit	slowapi 0.1.9	Pembatasan laju permintaan

3.3 Arsitektur Keamanan Sistem

Arsitektur keamanan TUMBUH menerapkan model AAA secara menyeluruh dengan dukungan layanan *SecurityService* pada *backend* dan mikroservis audit yang terpisah. *Frontend* menyimpan JWT dan mengirimkannya melalui tajuk *Authorization: Bearer* pada setiap permintaan. *Backend* menangani seluruh logika bisnis dan keamanan, termasuk enkripsi tingkat kolom serta penandatanganan peristiwa. Layanan audit menerima peristiwa dari *backend* secara *fire-and-forget* dan menuliskannya ke berkas *log* yang dirantai

dengan *hash*. Gambaran umum arsitektur disajikan pada Gambar 1.



Gambar 1 Arsitektur keamanan aplikasi TUMBUH yang mencakup data *at rest*, *in transit*, dan manajemen kunci

3.4 Pemodelan Ancaman dan Pemetaan Kontrol

Berdasarkan analisis aset dan ancaman, dirumuskan pemetaan kontrol keamanan terhadap risiko utama yang dihadapi TUMBUH. Pemetaan ringkas disajikan pada Tabel 2 dan menjadi dasar penentuan modul yang diimplementasikan.

Tabel 2 Pemetaan ancaman terhadap kontrol keamanan pada TUMBUH

Ancaman	Kontrol keamanan
Pencurian akun dan tebak paksa kata sandi	<i>Password hashing</i> bcrypt, <i>rate limiting</i> , verifikasi surel
Akses tidak sah ke data milik pengguna lain	RBAC, pemeriksaan kepemilikan, kontrol akses CV
Kebocoran data sensitif (surat lamaran)	Enkripsi tingkat kolom Fernet (AES-128-CBC + HMAC)
Manipulasi dan penyangkalan perubahan status	Tanda tangan digital Ed25519, jejak audit
Pemalsuan atau penghapusan catatan audit	Rantai <i>hash</i> SHA-256 (<i>tamper evidence</i>)
Kebocoran rahasia dan kunci kriptografi	Manajemen rahasia berbasis variabel lingkungan

3.5 Manajemen Kunci

Seluruh kunci kriptografi dan rahasia dikelola melalui variabel lingkungan dan tidak pernah ditanamkan langsung pada kode (*no hardcoded keys*). Kunci penting meliputi SECRET_KEY untuk penandatanganan JWT, FIELD_ENCRYPTION_KEY untuk kunci Fernet, dan DIGITAL_SIGNATURE_PRIVATE_KEY untuk benih kunci privat Ed25519. Pada

lingkungan pengembangan, kunci dapat diturunkan secara deterministik dari `SECRET_KEY` menggunakan SHA-256, sedangkan pada lingkungan produksi kunci disuntikkan secara eksplisit melalui variabel lingkungan. Berkas konfigurasi yang memuat rahasia dikecualikan dari repositori melalui `.gitignore`.

3.6 Prosedur Pengujian

Pengujian dilakukan menggunakan kerangka *unit testing* pytest terhadap fungsi-fungsi keamanan inti. Skenario pengujian dirancang untuk memverifikasi kerahasiaan, integritas, dan otorisasi, yang mencakup: (1) *roundtrip* enkripsi-dekripsi data sensitif; (2) deteksi pemalsuan pada tanda tangan digital; (3) penolakan akses oleh pengguna tanpa hak (mengharapkan HTTP 403); (4) penolakan dekripsi terhadap *ciphertext* yang dirusak; serta (5) pengujian keamanan akun seperti penonaktifan akun dan penggantian kata sandi. Setiap pengujian dijalankan secara terisolasi menggunakan basis data SQLite *in-memory* sehingga tidak bergantung pada infrastruktur produksi. Keluaran pengujian dicatat sebagai berkas *log* dan disertakan pada lampiran.

IV HASIL DAN PEMBAHASAN

Bab ini memaparkan hasil implementasi kontrol keamanan pada aplikasi TUMBUH beserta pembahasannya. Pemaparan diorganisasi mengikuti kerangka AAA, dilanjutkan dengan *password hashing*, enkripsi data, tanda tangan digital, kontrol pendukung, integrasi pada alur utama, dan hasil pengujian.

4.1 Authentication

Autentikasi pada TUMBUH mendukung tiga jalur, yaitu pendaftaran dan masuk berbasis surel dan kata sandi, autentikasi Google OAuth, serta pengelolaan sesi berbasis JWT. Pada pendaftaran, sistem memvalidasi format surel, memeriksa domain institusional (mahasiswa wajib menggunakan surel @ipb.ac.id), memeriksa duplikasi, melakukan *hashing* kata sandi, lalu menerbitkan token verifikasi surel. Pada proses masuk, sistem memverifikasi kata sandi, memastikan surel telah terverifikasi dan akun aktif, kemudian menerbitkan *access token* (berlaku 60 menit) dan *refresh token* (berlaku tujuh hari).

Token diterbitkan dan diverifikasi menggunakan algoritma HS256. Setiap permintaan ke sumber daya terlindungi mengekstraksi pengguna dari token melalui mekanisme *dependency injection* FastAPI. Sistem juga memastikan *refresh token* tidak dapat digunakan sebagai *access token* untuk memanggil API. Autentikasi Google dilakukan dengan memverifikasi tanda tangan, audiens, dan masa berlaku token identitas yang diterbitkan Google sebelum identitas dipercaya.

4.2 Authorization

Otorisasi diterapkan dengan model RBAC atas tiga peran, yaitu *student*, *hr*, dan *admin*. Pembatasan akses diwujudkan melalui *factory* dependensi *require_role()* yang menolak permintaan dengan status HTTP 403 apabila peran pengguna tidak sesuai, sebagaimana ditunjukkan pada Kode IV.1.

```

1 def require_role(required_role: str):
2     def _check_role(current_user: User = Depends(get_current_user)) -> User:
3         if current_user.role.value != required_role:
4             raise HTTPException(
5                 status_code=status.HTTP_403_FORBIDDEN,
6                 detail=f"Access denied. Required role: {required_role}",
7             )
8         return current_user
9     return _check_role

```

Kode IV.1 Dependensi RBAC pada backend

Selain pembatasan berbasis peran, sistem menerapkan pemeriksaan kepemilikan

(*ownership check*). HR hanya dapat mengelola pelamar pada lowongan milik perusahaan sendiri, baik pada pembaruan status tunggal maupun massal. Akses terhadap dokumen CV dibatasi sehingga hanya pemilik, admin, atau HR yang relevan (yang kepada perusahaannya mahasiswa melamar) yang dapat mengaksesnya. Dokumen CV disimpan pada direktori privat dan tidak pernah dilayani sebagai berkas statis, sehingga aksesnya selalu melewati lapisan autentikasi dan otorisasi.

4.3 Accounting (Audit Logging)

Akuntansi diwujudkan melalui mikroservis audit berbasis Express dan Winston yang menerima peristiwa dari *backend*. Pemanggilan audit bersifat *fire-and-forget* pada utas latar belakang sehingga tidak memperlambat respons API, bersifat *fault-tolerant* apabila layanan audit tidak tersedia, dan dibatasi *timeout*. Peristiwa yang dicatat mencakup registrasi, masuk berhasil dan gagal, penyegaran token, pengiriman dan pembaruan lamaran, pelampauan *rate limit*, serta kesalahan tak tertangani.

Integritas catatan audit dijamin dengan rantai *hash* SHA-256. Setiap peristiwa baru di-*hash* bersama *hash* peristiwa sebelumnya, sehingga pemalsuan atau penghapusan satu catatan akan memutus rantai dan dapat dideteksi. Mekanisme ini ditunjukkan pada Kode IV.2. Sistem menyediakan fungsi verifikasi rantai yang membaca seluruh berkas *log* secara kronologis dan memeriksa kecocokan setiap tautan *hash*.

```

1 function chainAuditEvent(event) {
2   const previousHash = loadLastHash();           // 'GENESIS' jika pertama
3   const eventHash = crypto
4     .createHash('sha256')
5     .update(previousHash + ':' + canonicalJson(event))
6     .digest('hex');
7   return { ...event, previousHash, eventHash,
8             integrityAlgorithm: 'SHA-256 hash chain' };
9 }

```

Kode IV.2 Perantain hash SHA-256 pada layanan audit

4.4 Password Hashing dan Salting

Kata sandi tidak pernah disimpan dalam bentuk asli. Sistem menggunakan *bcrypt* dengan *salt* acak yang dibangkitkan otomatis dan faktor biaya bawaan 12 putaran, sebagaimana ditunjukkan pada Kode IV.3. *Salt* ditanamkan langsung pada keluaran *hash*, sehingga dua kata sandi identik tetap menghasilkan *hash* yang berbeda. *Field* *hashed_password* tidak pernah dideklarasikan pada skema respons sehingga otomatis dikecualikan dari seluruh respons API.

```

1 @staticmethod
2 def hash_password(password: str) -> str:
3     return _bcrypt.hashpw(
4         password.encode("utf-8"), _bcrypt.gensalt()
5     ).decode("utf-8")
6
7 @staticmethod
8 def verify_password(plain: str, hashed: str) -> bool:
9     return _bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))

```

Kode IV.3 Hashing dan verifikasi kata sandi dengan bcrypt

4.5 Enkripsi dan Dekripsi Data

Kerahasiaan data sensitif, khususnya surat lamaran, dijaga dengan enkripsi tingkat kolom menggunakan Fernet. Fernet mengombinasikan AES-128 mode CBC untuk kerahasiaan dan HMAC-SHA256 untuk integritas, serta menyertakan *initialization vector* acak dan *timestamp* pada tiap token. Fungsi enkripsi dan dekripsi ditunjukkan pada Kode IV.4. Nilai terenkripsi diberi awalan `enc:v1:` untuk menandai status enkripsi, mencegah enkripsi ganda, dan memudahkan migrasi algoritma di masa depan.

```

1 def encrypt_text(self, value: str | None) -> str | None:
2     if value is None or value.startswith(ENCRYPTED_PREFIX):
3         return value
4     token = self._fernet.encrypt(value.encode("utf-8")).decode("utf-8")
5     return f"{ENCRYPTED_PREFIX}{token}"
6
7 def decrypt_text(self, value: str | None) -> str | None:
8     if value is None or not value.startswith(ENCRYPTED_PREFIX):
9         return value
10    token = value.removeprefix(ENCRYPTED_PREFIX).encode("utf-8")
11    try:
12        return self._fernet.decrypt(token).decode("utf-8")
13    except InvalidToken:
14        return "[encrypted value could not be decrypted]"

```

Kode IV.4 Enkripsi tingkat kolom dengan Fernet

Di basis data, surat lamaran tersimpan sebagai *ciphertext* berawalan `enc:v1:`, sedangkan pada respons API nilai dikembalikan sebagai teks biasa setelah didekripsi. Dengan demikian, kebocoran basis data tidak serta-merta mengungkap isi surat lamaran karena penyerang tetap memerlukan kunci enkripsi.

4.6 Tanda Tangan Digital dan Non-Repudiation

Peristiwa penting pada siklus lamaran ditandatangani secara digital menggunakan Ed25519 untuk menjamin integritas dan *non-repudiation*. Sebelum ditandatangani, *payload* dikanonikalisasi menjadi JSON dengan kunci terurut dan tanpa spasi berlebih agar bersifat deterministik. *Payload*, tanda tangan, dan nama algoritma disimpan pada basis data. Verifikasi dilakukan melalui *endpoint* khusus admin yang membandingkan tanda tangan dengan *payload* tersimpan, sebagaimana ditunjukkan pada Kode IV.5.

```

1 def sign_payload(self, payload: dict[str, Any]) -> str:
2     canonical = self._canonical_json(payload).encode("utf-8")
3     signature = self._private_key.sign(canonical)
4     return base64.urlsafe_b64encode(signature).decode("ascii")
5
6 def verify_signature(self, payload: dict[str, Any], signature: str) -> bool:
7     try:
8         raw = base64.urlsafe_b64decode(signature.encode("ascii"))
9         self._public_key.verify(raw, self._canonical_json(payload).encode("utf
10        -8"))
11         return True
12     except Exception:
13         return False

```

Kode IV.5 Penandatanganan dan verifikasi payload dengan Ed25519

Dengan mekanisme ini, mahasiswa tidak dapat menyangkal pengiriman lamaran dan HR tidak dapat menyangkal perubahan status pelamar karena *payload* memuat identitas pelaku dan *timestamp*. Apabila *payload* diubah setelah ditandatangani, verifikasi akan gagal.

4.7 Kontrol Keamanan Pendukung

Selain kontrol inti, TUMBUH menerapkan sejumlah kontrol pendukung. *Security headers* (X-Content-Type-Options, X-Frame-Options, dan Referrer-Policy) ditambahkan pada setiap respons untuk memitigasi *MIME sniffing* dan *clickjacking*. *Rate limiting* berbasis alamat IP membatasi *endpoint* autentikasi (5–10 permintaan per menit) untuk mencegah tebak paksa dan penyalahgunaan. CORS dibatasi pada daftar *origin* yang diizinkan. Unggahan berkas divalidasi berdasarkan tipe MIME dan ukuran, dengan nama berkas yang dibangkitkan ulang. Verifikasi surel menggunakan token sekali pakai yang disimpan dalam bentuk *hash* SHA-256 dan kedaluwarsa dalam 24 jam. Pencegahan injeksi SQL diperoleh melalui kueri terparameter pada ORM SQLAlchemy, sedangkan validasi masukan dijamin oleh skema Pydantic pada setiap *endpoint*.

4.8 Integrasi pada Alur Utama Aplikasi

Seluruh modul kriptografi terintegrasi langsung pada alur utama aplikasi, bukan sebagai skrip terpisah. Pada satu alur pengiriman lamaran, sistem secara berurutan melakukan validasi token dan peran, enkripsi surat lamaran sebelum disimpan, penandatanganan peristiwa, pencatatan audit, lalu dekripsi untuk respons. Keterpaduan ini memenuhi klausul integrasi yang mensyaratkan modul keamanan menyatu dengan *workflow* utama. Ringkasan fitur keamanan beserta teknologinya disajikan pada Tabel 3.

Tabel 3 Ringkasan fitur keamanan TUMBUH dan teknologi penyusunnya

Aspek keamanan	Teknologi
<i>Password hashing</i>	bcrypt (<i>auto-salt</i> , 12 putaran)
Autentikasi token	JWT HS256 (<i>access + refresh</i>)
Autentikasi pihak ketiga	Google OAuth (verifikasi token identitas)
Otorisasi	RBAC + pemeriksaan kepemilikan + kontrol CV
Enkripsi data	Fernet (AES-128-CBC + HMAC-SHA256)
Tanda tangan digital	Ed25519 pada peristiwa lamaran
Audit dan integritas catatan	Winston + rantai <i>hash</i> SHA-256
Kontrol pendukung	<i>Security headers</i> , <i>rate limit</i> , CORS, validasi unggahan
Manajemen rahasia	Variabel lingkungan, tanpa <i>hardcoded keys</i>

4.9 Hasil Pengujian

Pengujian unit dijalankan terhadap fungsi-fungsi keamanan menggunakan pytest. Seluruh sembilan kasus uji berhasil (*passed*). Ringkasan hasil disajikan pada Tabel 4, sedangkan keluaran lengkap pelaksanaan pengujian disertakan pada Lampiran 1 dan berkas *log* pada repositori.

Tabel 4 Ringkasan hasil pengujian fungsi keamanan

Kasus uji	Hasil
<i>Roundtrip</i> enkripsi-dekripsi surat lamaran	Lulus
Deteksi pemalsuan pada tanda tangan digital	Lulus
Penonaktifan akun memblokir proses masuk berikutnya	Lulus
Penghapusan akun memerlukan kecocokan surel	Lulus
Penggantian kata sandi melalui <i>reset</i> surel	Lulus
Pengguna student ditolak pada sumber daya admin (403)	Lulus
Pengguna admin diizinkan pada sumber daya admin	Lulus
Data terenripsi tidak terbaca sebagai teks biasa	Lulus
<i>Ciphertext</i> yang dirusak ditolak saat dekripsi	Lulus

Hasil pengujian membuktikan bahwa kontrol kerahasiaan, integritas, dan otorisasi bekerja sesuai rancangan. Pengujian penolakan akses (HTTP 403) dan penolakan dekripsi *ciphertext* yang dirusak secara khusus menunjukkan bahwa pengguna tanpa hak tidak dapat mengakses sumber daya terbatas dan bahwa modifikasi tidak sah terhadap data terenripsi dapat dideteksi. Dengan demikian, sasaran keamanan informasi pada TUMBUH telah terpenuhi pada cakupan yang diuji.

V SIMPULAN DAN SARAN

5.1 Simpulan

Implementasi keamanan informasi pada aplikasi TUMBUH telah berhasil menerapkan kerangka AAA secara menyeluruh dan terintegrasi pada alur utama aplikasi. Autentikasi diwujudkan melalui kata sandi ber-*hash* bcrypt, JWT, dan Google OAuth; otorisasi melalui RBAC yang dilengkapi pemeriksaan kepemilikan dan kontrol akses CV; serta akuntansi melalui mikroservis audit dengan rantai *hash* SHA-256. Kerahasiaan data sensitif dijaga dengan enkripsi tingkat kolom Fernet, sementara integritas dan *non-repudiation* pada peristiwa lamaran dijamin dengan tanda tangan digital Ed25519. Seluruh kunci dikelola melalui variabel lingkungan tanpa *hardcoded keys*. Pengujian unit terhadap fungsi keamanan menunjukkan seluruh kasus uji berhasil, termasuk skenario penolakan akses oleh pengguna tanpa hak dan penolakan dekripsi terhadap *ciphertext* yang dirusak. Dengan demikian, sasaran kerahasiaan, integritas, dan akuntabilitas pada cakupan yang diuji telah terpenuhi.

5.2 Saran

Beberapa saran untuk pengembangan lebih lanjut adalah sebagai berikut. Pertama, penerapan rotasi kunci (*key rotation*) secara berkala melalui mekanisme versi awalan `enc:vN`: agar transisi algoritma dan kunci lebih mudah dikelola. Kedua, perluasan cakupan enkripsi tingkat kolom pada atribut sensitif lain seperti data kontak dan profil. Ketiga, penambahan autentikasi dua faktor (*two-factor authentication*) untuk memperkuat perlindungan akun. Keempat, perluasan cakupan pengujian dengan *integration testing* dan *penetration testing* untuk menilai ketahanan sistem secara menyeluruh. Kelima, pemantauan dan *alerting* otomatis atas anomali pada catatan audit agar respons terhadap insiden menjadi lebih cepat.

DAFTAR PUSTAKA

- Bernstein D, Duif N, Lange T, Schwabe P, Yang B. 2012. High-speed high-security signatures. *J Cryptogr Eng.* 2(2):77–89. doi:10.1007/s13389-012-0027-1.
- Hardt D. 2012. The oauth 2.0 authorization framework. RFC 6749, Internet Engineering Task Force.
- Jones M, Bradley J, Sakimura N. 2015. Json web token (jwt). RFC 7519, Internet Engineering Task Force.
- National Institute of Standards and Technology. 2001. Advanced encryption standard (aes). Technical Report FIPS PUB 197, NIST.
- Provos N, Mazieres D. 1999. A future-adaptable password scheme. Technical report, USENIX Annual Technical Conference. FREENIX Track.
- Python Cryptographic Authority. 2024. Fernet (symmetric encryption). [internet]. [diacu 2026 Jun 20]. Tersedia dari: <https://cryptography.io/en/latest/fernet/>.
- Sandhu R, Coyne E, Feinstein H, Youman C. 1996. Role-based access control models. *Computer.* 29(2):38–47. doi:10.1109/2.485845.
- Stallings W, Brown L. 2017. *Computer Security: Principles and Practice*. Ed ke-4. Boston (US), Pearson.
- Vollbrecht J, Calhoun P, Farrell S, Gommans L, Gross G, de Bruijn B, de Laat C, Holdrege M, Spence D. 2000. Aaa authorization framework. RFC 2904, Internet Engineering Task Force.

LAMPIRAN

Lampiran 1 Keluaran pelaksanaan pengujian unit fungsi keamanan (pytest)

```

=====
TUMBUH - Security Unit Test Log
Mata Kuliah Keamanan Informasi (KOM1315) - Kelompok 9
Host: Linux x86_64 | Python 3.10.12 | pytest 9.1.1
=====

### Suite 1: test_security_service.py + test_account_security.py

collected 5 items
test_security_service.py::test_encrypt_decrypt_text_roundtrip PASSED
test_security_service.py::test_digital_signature_detects_tampering PASSED
test_account_security.py::test_deactivate_account_blocks_future_login PASSED
test_account_security.py::test_delete_account_requires_matching_email PASSED
test_account_security.py::test_password_reset_email_allows_change PASSED
===== 5 passed in 3.67s =====

### Suite 2: test_authorization_and_encryption.py
###      (Authorization 403 + data terenkripsi)

collected 4 items
[AUTHZ] admin -> admin resource : DIIZINKAN 200 PASSED
[CONF ] plaintext disembunyikan; ciphertext = enc:v1:gAAAAAB... PASSED
[AUTHZ] student -> admin resource : DITOLAK 403
        (Access denied. Required role: admin) PASSED
[INTEG] ciphertext dirusak -> dekripsi gagal:
        '[encrypted value could not be decrypted]' PASSED
===== 4 passed in 1.64s =====

TOTAL: 9 passed, 0 failed.

```

Lampiran 2 Struktur folder repositori proyek sesuai instruksi pengelolaan repositori Git

```
KOM1315_SmtGenap26_Kelompok9_TUMBUH/  
|-- 01_Proposal_&_Analisis/      # Proposal teknis & threat modeling  
|-- 02_Design_Documents/        # ERD, arsitektur, rencana uji  
|-- 03_Source_Code/  
|   |-- backend/                 # Modul kriptografi (encrypt/decrypt)  
|   |-- database/                # Skema DB & hashing/salting  
|   |-- digital_signature/       # Tanda tangan digital (Ed25519)  
|   '-- src/auth/                # Authentication & RBAC  
|-- 04_Reports_&_Paper/  
|   |-- Monitoring_P7/           # Laporan kemajuan Bab 1-3  
|   |-- Final_Technical_Report/  # Laporan teknis akhir (dokumen ini)  
|   '-- Scientific_Paper/        # Paper ilmiah format JIKA  
|-- 05_Testing_Logs/            # Log pengujian fungsi keamanan  
'-- README.md
```